

Theoretical Assignment – Question 4

PySpark Partitioning, Bucketing, DataFrame Operations, Tables & expr()

1. Partitioning and Bucketing in PySpark

Partitioning and bucketing are techniques used to organize data so Spark can process large datasets efficiently. They solve different problems and should not be treated as the same concept.

Partitioning

Partitioning divides data into separate directories based on the value of one or more columns.

Consider a Retail/Sales DataFrame containing order_id, customer_id, city and amount. If we partition by city, Spark can physically organize the output into directories such as city=Mysuru, city=Bengaluru and city=Chennai.

```
sales_df.write \
    .partitionBy("city") \
    .parquet("sales_output")

sales_output/
■
■■■ city=Chennai/
■    ■■■ part-....parquet
■■■ city=Bengaluru/
■    ■■■ part-....parquet
■■■ city=Mysuru/
    ■■■ part-....parquet
```

If a query filters on city, Spark can use **partition pruning** to avoid reading unrelated partitions.

```
sales_df.filter(
  sales_df.city == "Mysuru"
)
```

Sales Data

■		
■■■	Chennai	X
■■■	Bengaluru	X
■■■	Mysuru	✓

Partitioning is useful when a column is frequently used for filtering and has a reasonable number of distinct values. Common examples include date, year, month, country, region and city.

```
sales_df.write \
    .partitionBy("year", "month") \
    .parquet("sales")
```

Important: Avoid blindly partitioning by very high-cardinality columns such as `order_id` or `transaction_id` because this can create a very large number of directories/files.

Bucketing

Bucketing distributes rows into a fixed number of buckets based on the hash of one or more columns.

[illegible]

Here, **10 buckets** are specified. Rows are deterministically distributed among those buckets using the bucketing column.

Bucketing can be useful when a table is frequently involved in operations such as JOIN or GROUP BY on the bucket column. In suitable situations, Spark can reduce the amount of data shuffled between executors.

Partitioning: Which directory should this data go into, based on its partition value?
Bucketing: Which fixed bucket should this row go into, based on hashing?

Partitioning vs Bucketing

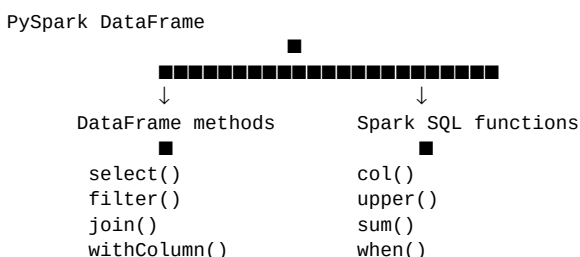
Feature	Partitioning	Bucketing
Basic idea	Creates directories based on column values	Distributes rows into a fixed number of buckets
Physical organization	Directory-based	Bucket/file-based
Number of groups	Depends on distinct partition values	Fixed number specified by you
Typical benefit	Partition pruning	Can reduce shuffle for suitable joins/aggregations
Common use	Filtering	JOIN / GROUP BY
API	partitionBy()	bucketBy()

Partitioning and bucketing can also be combined when the data layout and workload justify it.

```
sales_df.write \
    .partitionBy("year") \
    .bucketBy(10, "customer_id") \
    .saveAsTable("sales")
```

2. Difference Between Spark SQL Functions and DataFrame Operations

Spark SQL functions and DataFrame operations are not completely separate concepts. They commonly work together in PySpark.



DataFrame operations are methods used to perform operations on a DataFrame, such as `select()`, `filter()`, `join()`, `groupBy()` and `withColumn()`.

Spark SQL functions provide functions and column expressions such as `col()`, `upper()`, `sum()`, `avg()`, `when()` and `lit()`.

Example

```
from pyspark.sql.functions import col

df = df.withColumn(
    "gross_amount",
    col("quantity") * col("unit_price")
)
```

Here, **withColumn()** is the DataFrame operation, while **col()** creates a Spark Column expression. They work together to define the transformation.

```
from pyspark.sql.functions import upper, col

df.select(
    upper(col("customer_name")).alias("customer_name")
)
```

Memory model: DataFrame methods describe the operation on the DataFrame; Spark SQL functions help build the column expressions used by those operations.

3. How to Create a Table from a DataFrame

A DataFrame can be written as a table using **saveAsTable()**.

```
sales_df.write.saveAsTable("sales")
```

After the table is created, it can be queried using Spark SQL.

```
spark.sql(
```

```
"SELECT * FROM sales"
).show()
```

You can also specify a write mode.

```
sales_df.write \
    .mode("overwrite") \
    .saveAsTable("sales")

sales_df.write \
    .mode("append") \
    .saveAsTable("sales")
```

The first example overwrites the existing table/output according to the write operation; the second appends data to an existing table.

Creating a partitioned table

```
sales_df.write \
    .partitionBy("city") \
    .saveAsTable("sales")
```

Creating a bucketed table

```
sales_df.write \
    .bucketBy(10, "customer_id") \
    .saveAsTable("sales_bucketed")
```

```
DataFrame
  ↓
write
  ↓
saveAsTable()
  ↓
Spark Table
  ↓
spark.sql()
```

4. Understanding the `expr()` Function in PySpark

The **`expr()`** function allows us to use a **SQL-style expression** inside a PySpark DataFrame operation. It does not mean that an entire SQL query such as `SELECT ... FROM ...` is placed inside `expr()`; it is primarily used for expressions.

Example: Calculate gross amount

Suppose `quantity = 2` and `unit_price = 500`. We want `gross_amount = 1000`.

```
from pyspark.sql.functions import expr

df = df.withColumn(
    "gross_amount",
    expr("quantity * unit_price")
)
```

The text **`quantity * unit_price`** is written using SQL-style expression syntax.

The same operation using PySpark Column syntax

```
from pyspark.sql.functions import col

df = df.withColumn(
    "gross_amount",
    col("quantity") * col("unit_price")
)
```

Both approaches express the same transformation. The main difference is the syntax used to build the expression.

```
expr("quantity * unit_price")
    VS
col("quantity") * col("unit_price")
```

Another example: `UPPER()`

```
# SQL-style expression
df.select(
    expr("UPPER(customer_name)")
)

# PySpark Column syntax
```

```
from pyspark.sql.functions import upper, col

df.select(
    upper(col("customer_name"))
)
```

A useful CASE expression example

```
# Using expr()
df.withColumn(
    "category_type",
    expr("""
        CASE
            WHEN quantity >= 10 THEN 'Bulk'
            WHEN quantity >= 5 THEN 'Medium'
            ELSE 'Small'
        END
    """)
)

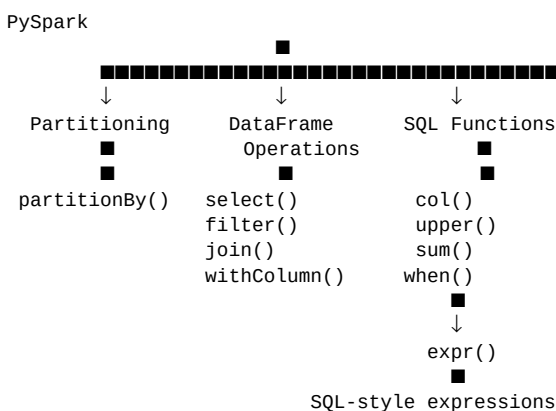
# Equivalent PySpark syntax
from pyspark.sql.functions import when, col

df.withColumn(
    "category_type",
    when(col("quantity") >= 10, "Bulk")
    .when(col("quantity") >= 5, "Medium")
    .otherwise("Small")
)
```

This illustrates why `expr()` can be convenient when a transformation is naturally expressed using SQL syntax, while the `DataFrame/Column` API provides a programmatic PySpark alternative.

Core idea: `expr("SQL-style expression")` and a PySpark Column expression can represent the same transformation. The difference is mainly the expression syntax.

Overall Mental Model



Important Points to Remember

- **Partitioning** → organizes output into directories based on partition column values and can enable partition pruning.
- **Bucketing** → distributes rows into a fixed number of buckets using a bucketing column and can help suitable JOIN/GROUP BY workloads.
- Partitioning is commonly associated with filtering; bucketing is commonly associated with repeated joins or aggregations on a bucket column.
- DataFrame methods such as `select()`, `filter()`, `join()` and `withColumn()` work together with Spark SQL functions such as `col()`, `upper()`, `sum()` and `when()`.
- **`saveAsTable()`** writes a DataFrame as a table that can be queried with Spark SQL.
- **`expr()`** accepts SQL-style expressions inside PySpark DataFrame operations.
- For `expr("quantity * unit_price")`, the PySpark Column equivalent is `col("quantity") * col("unit_price")`.